Fast Complete BN254 Scalar Multiplication

Twelve optimisations for ArgoMAC, with specification, security, and speedup

Kriterion research submission

6 September 2026

Abstract

This report presents twelve changes to the ArgoMAC garbled circuit for BN254 G1 scalar multiplication. Each change has its own section. Each section states whether the change alters the specification, gives the construction detail, gives the security analysis, and reports the measured speedup. Three changes are in the Lean construction: one shared garbling pass, a proved 91-digit recoding, and complete Renes–Costello–Batina addition. Nine changes are in the Rust implementation. The Lean benchmark median falls from 989 ms to 231 ms. The Rust protocol encode mean falls from 81.309 ms to 68.190 ms. Lean 4.24 checks correctness, the digit bound, the complete law, the topology, and the Lamport label order without a new axiom. The adaptive-privacy theorem is not complete; Section 15 states its exact status.

1 Summary

Table 1 lists every change. The Lean column refers to the challenge construction and its proof. The Rust column refers to the BaBe implementation. A specification change is a change to the paper construction, the wire format, the randomness derivation, or the evaluator acceptance rule. A change without a specification change computes the same function with the same public values.

§	Change	Layer	Spec change	Measured effect
2	Shared garbling pass	Lean	No	Lean 989 to 160 ms
3	Four-corner 91-digit recoding	Lean, Rust	Yes: digit count	One fewer output row
4	Complete RCB addition	Lean, Rust	Yes: row law, decode	Correctness; Lean 160 to 231 ms
5	Parallel rows, fixed RNG streams	Rust	Yes: RNG streams	Row 15.934 to 3.845 ms
6	Fixed AES window view reuse	Rust	No	Within noise
7	SHA-256 prefix reuse	Rust	No	Row 3.841 to 3.754 ms
8	Specialized hash-to-field reduction	Rust	No	One worker 20.858 to 19.906 ms
9	Four-limb field XOR	Rust	No	Row 3.713 to 3.390 ms
10	Direct hash-block reduction	Rust	No	Row 3.507 to 3.351 ms
11	One-block KDF input	Rust	Yes: KDF, version 5	Row 3.375 to 3.081 ms
12	Paired KDF digests	Rust	Yes: KDF, version 6	Row 3.121 to 2.876 ms
13	Noncanonical coordinate rejection	Rust	Yes: acceptance rule	None; hardening

Table 1: All changes. Each Rust row time is one adjacent A/B pair from that change's own benchmark run, so the baselines drift between rows.

All Rust times use an Apple M5 with ARMv8 AES, Rust 1.94.0, and 100 Criterion samples. Row times use ten Rayon workers unless the text says one worker. All Lean times are 21-run medians of the direct-table benchmark on the same machine.

2 Shared garbling pass

2.1 Specification change

None. The logical circuit, the public table, the fixed-key oracle indices, the topology, the input labels, and the assumptions do not change.

2.2 Detail

The baseline executes the same pure garbling operations six times in its dominant layer. It computes each bit-adaptor pair twice. It computes each point row three times. The shared pass computes each bit pair once and each point row once. It then reuses the values. The fixed-permutation application count falls from 6,322,060 to 1,057,910 for the original 92-row, nine-adaptor circuit.

2.3 Security analysis

Lean proves that both changed functions equal their baseline definitions for all inputs. Thus, the public table and the evaluation function are identical. The adversary view does not change. No security argument depends on this change.

2.4 Speedup

The Lean median fell from 989 ms to 160 ms. This is a 6.18 times speedup. It is an 83.8 percent reduction. This measurement uses the original 92-row incomplete law, before Sections 3 and 4.

3 Four-corner 91-digit recoding

3.1 Specification change

Yes. The scalar decomposition uses 91 fixed positions instead of 92. The digit alphabet, the endomorphism, and the input-label shape do not change. The Rust constant `KAPPA` changes from 92 to 91. Old artifacts are not wire-compatible.

3.2 Detail

Let r be the BN254 scalar modulus. Let ω be the nontrivial cube root used by the G1 endomorphism. Set $\beta = 2 - \omega$. The digit alphabet is

$$D = \{0, \pm 1, \pm\omega, \pm\omega^2\}.$$

The norm of β is seven. Use the Eisenstein norm

$$Q(a, b) = a^2 - ab + b^2.$$

The relevant BN254 constants are

$$\begin{aligned} A &= 147946756881789319000765030803803410728, \\ d &= 9931322734385697763, \\ C &= A + d, \\ r &= A^2 + Ad + d^2. \end{aligned}$$

The two GLV kernel vectors are

$$v_1 = (-A, d), \qquad v_2 = (-d, -C).$$

Arkworks divides the two negative quotient numerators toward zero. Thus, its GLV state is the floor representative z_0 used by the proof. The selector checks

$$z_0, \quad z_0 + v_1, \quad z_0 + v_2, \quad z_0 + v_1 + v_2.$$

Each shift is in the reconstruction kernel, so every candidate represents the same scalar. For quotient remainders $u, v \in [0, r)$, scaling maps these four states to the four corners of one Eisenstein cell. Two triangle lemmas show that one corner satisfies $3Q(a, b) \le r$. This is the exact covering-radius step.

One recurrence round selects a unit digit and divides by β. For a state with $Q(a, b) \le m$, the selected digit gives

$$Q(a - u_d, b - v_d) \le m + 1 + \lfloor\sqrt{4m}\rfloor,$$

and exact division gives $Q(a - u_d, b - v_d) = 7Q(a', b')$. Define the inverse safe bound

$$\begin{aligned} B(0) &= 2, \\ B(n+1) &= 7B(n) + 1 - \left\lfloor\sqrt{4(7B(n) + 1)}\right\rfloor. \end{aligned}$$

Lean proves $Q(a, b) \le B(n) \implies \mathsf{after}(n + 1, (a, b)) = (0, 0)$. It also checks $r \le 3B(90)$ with ten-round kernel-checked checkpoints and no `native_decide`. The main Lean results are:

- `shiftedInitialCorrect`;

- `existsCandidateNormBound`;
- `fourCandidatesSignedI128`;
- `existsCorrectShiftedInitialTerminates91`;
- `shortInitialTerminates91`;
- `shortInitialCorrect`.

Lean also checks $7^{90} < r < 7^{91}$. A fixed 90-position string over seven symbols has at most 7^{90} values, so 91 is optimal for this alphabet.

3.3 Security analysis

Every scalar uses exactly 91 positions. Thus, the public circuit has 91 output rows for every scalar, and the topology does not reveal the scalar. The Rust selector always runs all four 91-round tests. It selects the first terminating candidate with `subtle::Choice` and has no secret-dependent early return. Lean proves that every tested coordinate fits a signed 128-bit integer, so the fixed-width arithmetic cannot overflow. The report does not claim compiler-level constant time. The recoding is a public deterministic function of the secret scalar with the same alphabet, so the hybrid proof structure of the paper does not change.

3.4 Speedup

The dominant layer has 91 rows instead of 92. This removes one row of 13 digit adaptors and 3,302 bit adaptors. The per-row cost is unchanged. The change also removes a sampled termination claim from the trust base and replaces it with a kernel-checked proof.

4 Complete homogeneous addition

4.1 Specification change

Yes. The point row uses Algorithm 7 of Renes, Costello, and Batina in homogeneous coordinates instead of the exceptional affine law. The row polynomials, the coefficient support flags, and the decode rule change. The Rust constant `PRF_VERSION` rises to 6 and binds the law into the setup commitment.

4.2 Detail

Let the hidden offset be $K = (A : B : C)$. Let (u, v) be the selected endomorphism image of the evaluator input. Use homogeneous coordinates for $Y^2Z = X^3 + 3Z^3$. An affine point is $(x : y : 1)$ and the identity is $(0 : 1 : 0)$. The three row polynomials are

$$\begin{aligned}
X &= -9AB - 18BCu - 18ACv + (B^2 - 9C^2)uv + ABv^2,\\
Y &= -81C^2 + 27A^2u + 27ACu^2 + B^2v^2,\\
Z &= 9BC + (B^2 + 9C^2)v + BCv^2 + 3A^2uv + 3ABu^2.
\end{aligned}$$

One fresh nonzero value ρ multiplies all three coordinates. The zero digit returns $\rho(A, B, C)$. The decoder keeps Z until the final projection. For $Z \neq 0$ it validates $(X/Z, Y/Z)$. For $Z = 0$ it accepts only $X = 0$ and $Y \neq 0$ and returns the identity. It rejects all other zero-denominator triples.

The old affine law fails perfect correctness. For scalar 2, output row 1 has digit `.one`. Select the valid input equal to that row offset. The old denominator is zero, the unchecked affine conversion becomes $(0, 0)$, and decoding fails.

Lean proves the homogeneous curve equation, that the output is nonzero and projectively nonsingular, and that projective decode equals Mathlib group addition for every valid affine pair. The proof has separate cases for different x-coordinates, doubling, and inverse inputs. The main results are `outputNonsingular`, `outputRepresentsSum`, `decodeHomogeneous_eq_toAffine`, `decodeEvaluateRowSome`, `decodeEvaluateRowNone`, and `perfectCorrectness`.

The complete Bosma–Lenstra family has projective parameters $[\alpha : \beta : \gamma]$ with $\beta \neq 0$. Normalize $\beta = 1$. Under the exact `Biquadratic` support rule the adaptor counts are $D(0, 0) = 13$, $D(0, \gamma) = 14$ for $\gamma \neq 0$, and

$D(\alpha, \gamma) = 15$ for $\alpha \neq 0$. Thus, the RCB parameters $[0 : 1 : 0]$ give the unique minimum of 13. A second bound covers private role-preserving linear rows. A row with t scalar adaptors exposes $14 + t$ selected field values and has adaptor randomness of dimension at most $2t - 2$. Only the three output coordinates can stay unmasked, so $2t - 2 \geq 11 + t$ and $t \geq 13$. A correct ten-adaptor sharing attempt violates this bound and exposes four extra coefficient invariants.

4.3 Security analysis

The change repairs perfect correctness. BN254 G1 has cofactor one and odd prime order, so it has no point of order two. The RCB exceptional line $Y = 0$ does not meet this group. Thus, the law covers doubling, inverse addition, and identity inputs for all valid G1 points. The common factor ρ randomizes the homogeneous representation, so the row does not reveal the offset scale. Every support flag is public and independent of the scalar. The $Z = 0$ branch of the decoder reveals only that the decoded output is the group identity. The 13-adaptor row is optimal in the fixed bidegree $(2, 2)$ family and in the private role-preserving linear model. The row exposes no coefficient function beyond the three outputs.

4.4 Speedup

This change increases work. Each row uses 13 digit adaptors instead of nine. Table 2 gives the exact counts. The Lean median rises from 160 ms to 231 ms. The complete circuit remains 4.28 times faster than the 989 ms baseline. The HMAC table grows by 2,970,240 bytes to 9,668,659 bytes.

Item	Original 92-row call	Complete 91-row call	Change
Output MAC rows	92	91	-1
Digit adaptors	833	1,188	$+355$
Bit adaptors	211,582	301,752	$+90,170$
Fixed-permutation applications	6,322,060	1,508,760	$-4,813,300$
Public permutation instances	12,495	17,820	$+5,325$

Table 2: Circuit cost. The first column counts repeated calls in the unoptimized baseline. The last column uses one shared pass for each bit and point row.

5 Parallel rows with fixed RNG streams

5.1 Specification change

Yes, in the randomness derivation. The wire format does not change. The garbler samples one fresh secret 32-byte root from the caller RNG. Row j uses ChaCha12 stream

$$\texttt{0x666d3265635f6d61} + j$$

from word position zero. The code allocates the three child nonce domains of every row before Rayon starts. The specification records this schedule.

5.2 Detail

The Rust implementation allocates one job for each of the 91 output rows. Release builds execute the jobs with Rayon. Debug builds use the same RNG and nonce schedule in serial order so the label-reuse diagnostic stays complete. Indexed collection keeps the row order. Jobs borrow the output keys and do not copy secret keys into a second vector. The root seed is erased after row construction. Fixed Boolean arrays replace heap-based coefficient index sets; this removes 27 net lines and 273 tree constructions per call.

5.3 Security analysis

Worker order cannot change the nonce domains, the caller RNG state, the row randomness, or the serialized row order. Tests compare one-worker and four-worker tables and the final caller RNG state. A second test compares all 273 allocated child nonces with the serial schedule. The field-row seed is secret. The pair of field-row seed and stream number must not repeat, and the security policy now states this requirement next to the existing seed, pre-seed, and nonce requirements. Under this requirement the row randomness is a fresh pseudorandom stream per row, which is the same assumption the serial code already makes on its single stream.

5.4 Speedup

The serial 92-row incomplete baseline row estimate is 15.934 ms. The complete 91-row, ten-worker row mean after this change is 3.845 ms. This is 4.14 times faster. A shared base row RNG did not improve this time and was discarded.

6 Fixed AES window view reuse

6.1 Specification change

None. The fixed AES keys, inputs, and outputs do not change.

6.2 Detail

Each digit adaptor creates one fixed AES window view for each 91-bit chunk instead of one view per bit. The view holds the 15 expanded keys of that window.

6.3 Security analysis

The change reorders allocation only. The same keys encrypt the same labels.

6.4 Speedup

The measured row mean changed from 3.845 ms to 3.841 ms. This is within noise. A full AES block batching variant was slower at 4.578 ms and was discarded.

7 SHA-256 prefix reuse

7.1 Specification change

None. Every derived key byte is identical.

7.2 Detail

The key derivation for the 15 fixed AES keys of one window shares one SHA-256 prefix for the bucket and role fields. The code hashes the prefix once and forks the state for each key.

7.3 Security analysis

A test compares every derived cipher with the original full KDF input. The output is the same function of the same input, so the key distribution does not change.

7.4 Speedup

The row mean fell from 3.841 ms to 3.754 ms. This is a 2.3 percent reduction.

8 Specialized hash-to-field reduction

8.1 Specification change

None. The 384-bit integer and its reduction modulo p do not change.

8.2 Detail

The reducer assembles the fixed 48-byte hash input as a 17-byte low part and a 31-byte high part directly into field limbs. It replaces the generic slice loop.

8.3 Security analysis

All 384 one-bit inputs and the prior edge vectors match the Arkworks reduction. A compile-time constant for 2^{136} in $\mathbb{F}_q$ did not help and was discarded.

8.4 Speedup

The one-worker row mean fell from 20.858 ms to 19.906 ms. This is a 4.6 percent reduction. The ten-worker row mean is 3.630 ms.

9 Four-limb field XOR

9.1 Specification change

None.

9.2 Detail

The 256-bit field plaintext is XORed into its pad as four explicit little-endian 64-bit limbs. This replaces nested base-field, limb, and byte loops.

9.3 Security analysis

XOR is bitwise, so the limb order changes no output bit. The ciphertext row is the same value.

9.4 Speedup

The row mean fell from 3.713 ms to 3.390 ms. Criterion reports an 8.7 percent improvement. A native-endian 128-bit Davies–Meyer feed-forward XOR was 5.5 percent slower and was discarded.

10 Direct hash-block reduction

10.1 Specification change

None.

10.2 Detail

The three AES hash blocks feed the field reduction directly. The code no longer copies them into a 48-byte staging array first.

10.3 Security analysis

The specialized limb split preserves every bit of the previous little-endian integer. The hash-to-field value is identical.

10.4 Speedup

The row mean fell from 3.507 ms to 3.351 ms. Criterion reports a 4.5 percent improvement. The one-worker row mean is 18.411 ms.

11 One-block KDF input

11.1 Specification change

Yes. The fixed-key KDF input packs the level, sub-bucket, role, and slot into one injective byte. The complete input is 54 bytes. `PRF_VERSION` 5 binds the new domain and schedule into the setup commitment.

11.2 Detail

The 54-byte input and its SHA-256 padding fit in one compression block. The metadata byte packs the level into bits 0–2 and the window-major key index into bits 3–5. A contiguous 54-byte buffer did not improve the benchmark further and was discarded.

11.3 Security analysis

The valid ranges make each digest input distinct. Domain separation is preserved because the version tag and the injective metadata map distinct keys to distinct random-oracle inputs. The KDF is modeled as a random oracle, so each key is an independent uniform value as before.

11.4 Speedup

The row mean fell from 3.375 ms to 3.081 ms. This is an 8.7 percent reduction. The one-worker row mean is 16.950 ms.

12 Paired KDF digests

12.1 Specification change

Yes. One SHA-256 digest supplies two 128-bit AES keys. Eight digests derive the 15 keys of one window. `PRF_VERSION` 6 and the domain tag `afk/6` bind this schedule.

12.2 Detail

For flat key index i, set $j = \lfloor i/2 \rfloor$ and $h = i \bmod 2$. Key i is the h-th 128-bit half of digest j. The second half of the last digest is unused. Skipping its AES key expansion did not change the benchmark and was not adopted.

12.3 Security analysis

The map from each flat key index to its digest pair and half is injective. In the random-oracle model both 128-bit halves of each 256-bit digest are independent uniform values. Thus, the 15 keys remain independent and uniform.

12.4 Speedup

The row mean fell from 3.121 ms to 2.876 ms. Criterion reports a 7.9 percent improvement. The one-worker row mean is 15.348 ms.

13 Noncanonical coordinate rejection

13.1 Specification change

Yes, in the evaluator acceptance rule. The checked evaluator rejects a 254-bit coordinate half that is not less than p before field reduction. It then checks the curve equation before it evaluates the HMAC table.

13.2 Detail

A regression test uses $x = p + 1$ and $y = 2$. Reduction would otherwise map this input to the valid generator $(1, 2)$. The Lean public wrapper uses the canonical `AffineInput` and proves that its 508 encoding labels match `affineLamportBits` in x-then-y order.

13.3 Security analysis

The signed raw bits, the proof-derived canonical binding, the checked gadget path, and the final message-hash check must all agree for decryption to succeed. A noncanonical encoding can no longer alias a different valid point.

13.4 Speedup

None. The change is a hardening.

14 End-to-end results

The complete row stage is 5.54 times faster than the serial baseline: 2.876 ms against 15.934 ms. The 95 percent interval of the final row mean is 2.861 to 2.893 ms. The protocol `encode` mean is 68.190 ms with a 95 percent interval of 67.963 to 68.581 ms. The comparable baseline is 81.309 ms. The complete implementation reduces this time by 16.1 percent. The Lean direct-table median is 231 ms against 989 ms, which is 4.28 times faster.

15 Formal verification status

This section states what Lean 4.24 checks, what remains open, and what the companion challenge change fixes. Every theorem named below passes the axiom audit with only `propext`, `Classical.choice`, and `Quot.sound`. The proof adds no `sorry`, no author axiom, and no `native_decide`.

15.1 Closed properties

- **Correctness.** `perfectCorrectness` proves that every valid canonical input and every successful garbling run decodes to the scalar multiple (Section 4).
- **Digit bound.** `shortInitialTerminates91` and `shortInitialCorrect` prove that the four-corner recoding terminates in 91 rounds and reconstructs the scalar (Section 3).
- **Topology.** Every scalar gives 91 output positions and 508 Lamport label positions, so the public topology is constant.
- **Lamport compatibility.** `Lamport.compatible` proves that the encoding key holds the 508 label pairs in x-then-y order and that encoding selects the label of each bit of `affineLamportBits`.
- **Adaptor optimum.** The 13-adaptor RCB row is minimal in the fixed bidegree $(2, 2)$ family and in the private role-preserving linear model (Section 4).

15.2 Adaptive privacy: proved components

The 100-bit adaptive-privacy theorem is not complete. Lean proves the generic reduction and most construction-specific components. The remaining gap is one distributional transport, stated in the next subsection.

Programming laws. Fresh permutation and random-oracle programming are involutions on a finite joint tape. Each map sends (O, t) to the programmed oracle and the old value $O(x)$, so each preserves the uniform joint PMF exactly. The permutation law composes over every fixed finite target-tape schedule. Both laws preserve the uniform fiber of oracles that match a fixed query transcript.

Adversary lift and coupling. Lean lifts an exact handler equivalence through every bounded adversary oracle program. A shared-sample coupling gives both traced marginals, equal results and query lists, and related final states. The bounded bridge form accepts a bridge that returns an exact result or records a bad event; it keeps both marginals exact by an independent coupling after a bad event. Every result disagreement implies the bad flag, and the final bad mass equals the bridge bad probability. A generic theorem reduces the complete `ConcreteAdaptivePrivacy` property to one trace transport and its Boolean change event.

Collision accounting. Each reached adversary query and its state is recorded, and the trace length is at most the query budget. A set of at most k forbidden blocks has mass at most $k \cdot 2^{-128}$, so a collision schedule over q queries has mass at most $qk \cdot 2^{-128}$. The 384-bit hash-to-field domain splits into $p\lfloor 2^{384}/p \rfloor$ complete fibers and one rejected suffix of mass at most $p/2^{384}$. A finite union bound over all 301,752 selected gates gives the aggregate rounding error $301752p/2^{384}$ without an independence assumption, and Lean proves that this term keeps 100 bits.

Gate programming. The gate programmer applies an ordered selected-gate schedule, preserves the shared invariant, and counts every programmed slot: three for a false bit and two for a true bit. Lean instantiates the schedule for one 254-bit digit adaptor, for one 13-adaptor RCB row with 3,302 gates, and for the full circuit with 1,188 adaptors and 301,752 gates. The linked schedule derives the point-layer input MAC with the exact EncPRF transform. A satisfied schedule equals the full pipeline evaluation and returns every requested homogeneous value. Retargeting puts one field residual in the low bit position of each selected row and preserves every public table. The ideal output vector puts the requested point in row zero and the identity in the other 90 rows, and Lean proves that it decodes to the requested point.

Simulator. The concrete online simulator returns the selected labels, retargets the curve and point requests, and applies the linked schedule. The offline coin is finite and contains no scalar or selected input. It samples every public coefficient, ciphertext row, label pair, permutation family, random oracle, and one hash-lift quotient per gate uniformly.

Distribution transports. Lean proves exact uniform laws for the following real values on the complete security tape:

- the three curve coefficients and all RCB X, Y, and Z coefficient groups, as affine bijections of their row masks;
- the 384-bit gate hash of one real gate, with the exact rejected-suffix mass;
- the 256-bit ciphertext row of one real bit adaptor, for keys and slopes that depend on all non-fixed-oracle randomness;
- the RCB zero pad on the good event, as the sum of the `y10` and `x9` accepted-residue digit offsets;
- the five fixed-key reads of one gate jointly, so the hash-based zero pad and the pad-based ciphertext row of one gate are independent;
- the joint reads of any injective family of fixed-key indices, with no bound on the family size.

15.3 Adaptive privacy: open gap

The construction still lacks the joint distributional hop that transports the real public garbling view to the offline coin outside the recorded collision event. The window-sharing optimization reads one fixed-key permutation at 91 labels per window, so the full read family is not index-injective. The transport therefore factors as independence across the 17,820 distinct window indices times the within-window collision that the existing bad event already bounds. The remaining proof must package each window's read as one value, instantiate the injective-family law across windows, and compose the result with the coefficient transports and the collision bound into the bridge premise.

The arithmetic side is closed. The worst selected branch uses the three hash slots and has zero-query numerator $3 \cdot 10692 \cdot 91^2 = 265621356 < 2^{28}$, which leaves a margin of 2,814,100 units. A direct five-slot union bound would exceed 2^{28}, so the proof must condition on the selected branch. Lean proves the 100-bit arithmetic for each exclusive three-hash or two-pad branch, and proves $3f^2 + 2t^2 \leq 3(f+t)^2$ and $3f + 2t \leq 3(f+t)$ for a bucket with f false and t true selections.

The formal property counts oracle queries and one decision step. It does not count adversary local computation or simulator work, so it is an ideal-oracle query metric rather than a standard PPT definition.

15.4 Challenge interface

A companion Kriterion change closes two interface defects. It makes `Kriterion.Solution` a closed dependent bundle and quantifies every proof over the BN254 field and group certificates, which remain declared challenge assumptions. It adds the certificate-free `Kriterion.Benchmark.garble` at a fixed scalar, security parameter, and seed, with a required equality to the proved scheme garbler. The security game samples the complete finite random tape uniformly; the fixed 256-bit seed selects only the reproducible benchmark tape. Until the adaptive proof is complete, no value inhabits `Kriterion.Solution`, and the challenge must reject this entry.

16 Reproduction

The Lean proof uses Lean 4.24 and the pinned Mathlib checkout. Run:

```
lake build Proof Tests
lake env lean entry/tests/Correctness.lean
lake env lean AxiomCheck.lean
lake exe bench
```

The Rust validation uses Rust 1.94.0. Run:

```
cargo fmt --all -- --check
cargo clippy --all-targets --all-features -- -D warnings
cargo test --all-features
cargo test --release --all-features
RAYON_NUM_THREADS=10 cargo bench --all-features \
  --bench field_mac_to_ec_mac -- 'field_mac_to_ec_mac/g1/garble'
RAYON_NUM_THREADS=10 cargo bench --all-features \
  --bench protocol_bench -- 'encode'
```

References

J. Bosma and H. W. Lenstra, “Complete systems of two addition laws for elliptic curves,” *Journal of Number Theory*, 1995.

J. Renes, C. Costello, and L. Batina, “Complete addition formulas for prime order elliptic curves,” EUROCRYPT 2016.

RustCrypto, *AES 0.8.4 documentation*, ARMv8 run-time detection section.

Arkworks, *ark-ec 0.5.0*, GLV scalar decomposition implementation.